



## VB Scripting – TEST

| Name | Date |
|------|------|
|      |      |

### Instructions

- Submit all sections via GitHub.
    - Section A should be a .vbs file.
    - Section B, answer all questions in a word document
  - Create a git repository and add both files (.vbs and document)
  - Submit your git repository link via email on or before due date.
- 

### Section A

Create a balance tracking program that consists of the following options in the main menu:

- 1 - Withdrawal
- 2 – Deposit
- 3 – Transfer
- 4 – Check balance
- 5 - Exit

At the start the user will be asked to capture their account number and then select one of the listed options. After selecting any one of the selected options, the user is navigated to the main menu, except if Exit is selected, and if Exit is selected then exits the program. Furthermore, for withdrawals and deposits, prompt the user to specify amount and decrease or increase the account balance respectively. For electronic transfers, prompt the user to specify the receiving account number and then the amount to transfer, then decrease the account balance. For each option, display the account holder, account number and remaining balance.

1. Create a class called **Account** with the following attributes:

Account number, Account holder, balance (The initial account balance is R1000)

The class should have the following sub routines or functions.

- **Withdrawal()** – this method takes amount as a parameter then subtracts the amount from the balance.



- **Deposit()** – this method takes amount as a parameter then adds the amount to the balance.
- **Transfer()** – this method takes account number (account number you want to transfer to) and amount to transfer as parameters, then subtract the amount from the balance.
- **CheckBalance()** – this method displays the balance on the message box.
- **getAccountNumber()** – This method takes no parameters and returns the account number as follows: **ACC-1234567**.
- **getAccountDetails()** - this method takes no parameters and returns the account holder, account number and remaining balance (nicely formatted).

## 2. Main program

Write the script that will execute the Account class

## 3. Validations

- Account number should be 7 - 10 digits long
- User cannot withdraw or deposit zero amount
- Users cannot withdraw more than their current balance
- Users cannot transfer more than their current balance
- Display an appropriate message should any of these validations be violated.

## Section B

### 1. Test Cases and Coverage

- 1.1. For each function and subroutine from your script, design test cases that will achieve 100% statement coverage and 100% branch coverage.

Use the table as a template.

| TC         | Input values                       | Statement Coverage % | Branch Coverage % |
|------------|------------------------------------|----------------------|-------------------|
| Function A |                                    |                      |                   |
| TC1        | Age = 18                           | 63%                  | 50%               |
| TC2        | Amount = R500<br>Status = employed | 84%                  | 75%               |
| Function A |                                    |                      |                   |
| TC1        |                                    | 33%                  | 25%               |

### 2. Equivalence Partitioning

Consider the following VBScript code and answer questions 1 and 2

```
age = InputBox("Enter your age")  
  
If age >= 18 And age <= 60 Then  
    MsgBox "Eligible"  
Else  
    MsgBox "Not Eligible"  
End If
```

- 2.1. Identify the valid and invalid equivalence partitions.  
2.2. Select one test value from each partition.  
2.3. State the expected result.

### 3. Boundary Value Analysis (Use the same script from 2)

- 3.1. Using 2-value Boundary Value Analysis, identify suitable test values.



#### 4. Decision Logic Table

Create a decision table based on the below script.

```
If username = "admin" And password = "admin123" Then
    MsgBox "Login Successful"
Else
    MsgBox "Login Failed"
End If
```

#### 5. Statement Coverage

Given this script

```
number = InputBox("Enter number")
```

```
If number > 0 Then
    MsgBox "Positive"
End If
```

```
MsgBox "Finished"
```

What is the minimum number of test cases required to achieve 100% statement coverage?

#### 6. Branch Coverage

Given this script

```
If number >= 10 Then
    MsgBox "Large"
Else
    MsgBox "Small"
End If
```

Provide value(s) required to design tests for **100% branch coverage**.

## 7. Error, Defect and Failure

Given this script

```
Function CalculateDiscount(amount)

    If amount >= 1000 Then
        CalculateDiscount = amount * 0.5
    Else
        CalculateDiscount = 0
    End If

End Function
```

Customers spending R1,000 or more receive a 5% discount.

**The code contains a defect.**

- 7.1. What is the defect?
- 7.2. Which test input exposes the defect?
- 7.3. What is the resulting failure?
- 7.4. Explain the process of fixing the defect. Also, include the type of tests that will be valuable.